

Как пользоваться проектом АРШИН

Привет. Если читаешь это — значит тебе достался проект и надо разобраться как он работает. Постарался расписать всё по-простому, без занудства.

Что вообще за проект

Скрипты ходят в публичный реестр ФГИС Аршин, забирают оттуда данные о поверках счётчиков воды и тепла и складывают в базу PostgreSQL. Дальше с этими данными уже можно работать как удобно.

Под капотом четыре скрипта, которые запускаются по очереди один за другим. Управляет ими файл `запуск.py` — это своеобразный пульт, через который удобно дёргать всё разом или по отдельности.

Что нужно чтобы это завелось

- **Python** — версия 3.10 или новее
- **PostgreSQL** — без разницы какой версии, лишь бы свежий
- **Интернет** — куда же без него, скрипты лезут в API
- **Любой редактор кода** — я работаю в PyCharm, но в принципе пофиг

Если чего-то нет — поставь, потом возвращайся.

Первый запуск — пошагово

Ставим зависимости

Открываешь терминал в папке проекта и пишешь:

```
pip install -r requirements.txt
```

Поставится несколько библиотек — `requests` чтобы лазить в API, `psycopg2-binary` для

PostgreSQL, `pdfplumber` для парсинга PDF и ещё пара мелочей. Минут пять подождать.

Создаём базу

Заходишь в свой PostgreSQL — через pgAdmin, DBeaver, psql или что у тебя там. Создаёшь новую базу, называешь как хочешь. Внутри неё ничего создавать не надо — все таблицы скрипты сами поднимут при первом запуске.

Прописываем подключение к базе

Это самое скучное. В каждом из четырёх скриптов в самом начале есть блок:

```
PG = dict(host="...", port=5432, dbname="...",
          user="...", password="...")
```

Подставляешь свои значения — адрес, имя базы, логин, пароль. Делать это нужно во всех четырёх файлах: `dimbrand.py`, `Обновлпервич.py`, `енрич.py`, `вольюмс.py`.

Если будешь публиковать код на GitHub или работать в команде — лучше пароли вынести в переменные окружения, а не оставлять прямо в коде. Это плохая практика, но для локальной работы и так сойдёт.

Проверяем что всё на месте

В папке должны быть пять скриптов: четыре рабочих и оркестратор `запуск.py`. Если чего-то не хватает — что-то пошло не так при клонировании.

Всё, можно запускать.

Как это устроено внутри

Если надо понять что вообще происходит — вот логика по шагам.

Шаг 0 — собираем бренды. Скрипт `dimbrand.py` берёт жёстко прописанный в коде список конкурентов и по каждому идёт в API за дополнительной инфой — стране и городу производителя. Кладёт всё в `dim_brands`. Запускается редко — только когда добавляешь новый бренд.

Шаг 1 — определяем кто реально производит. В реестре нет признака «вот это производитель, а это обслуживающая контора». Поэтому приходится определять самому. Логика такая: производители делают первичные проверки на заводе перед отгрузкой, а эксплуатанты — периодические каждые несколько лет. У каждой проверки

в API есть тип, по нему и считаем долю. Если у организации больше 70% первичных — записываем её как производителя в `manufacturers_verifiers`.

Шаг 2 — обогащаем данные о моделях. `енрич.ру` для каждой модели вытаскивает характеристики: какие у неё бывают диаметры, какая категория (бытовые, домовые, промышленные счётчики или теплосчётчики), и пару технических признаков. Главные источники — PDF с методиками поверки и реальные поверки. Всё складывается в `dim_reference`.

Шаг 4 — считаем объёмы. Финальный этап. `вольюмс.ру` идёт по каждому производителю и каждому месяцу года, спрашивает у API сколько было поверок и какого они типа. По диаметрам разносит цифры по категориям. Результат — таблица `production_volumes_exact`, в ней главные данные ради которых всё и делалось.

(Кстати, шага 3 нет — историческая нумерация. Не парься.)

Когда и что запускать

Если развернул проект с нуля

Открываешь `запуск.ру`, в начале файла все четыре флага должны быть `True`:

```
RUN_DIMBRAND = True
RUN_PERVICH  = True
RUN_ENRICH   = True
RUN_VOLUMES  = True
```

```
YEARS = [2023, 2024, 2025]
```

В `YEARS` ставишь те годы, за которые нужны данные. Жмёшь зелёную кнопку или запускаешь из терминала `python запуск.ру`.

Готовься ждать. Первый прогон долгий — может уйти несколько часов, зависит от количества брендов и объёма данных. Лучше ставить на ночь.

Если просто обновляешь данные раз в месяц

Это основной режим. Каждый месяц нужно подгружать новые поверки за текущий год. Полный прогон при этом запускать не нужно — справочники меняются редко.

В `запуск.ру` оставляешь только последний шаг включённым:

```
YEARS = [2025]
```

```
RUN_DIMBRAND = False
```

```
RUN_PERVICH = False
```

```
RUN_ENRICH = False
```

```
RUN_VOLUMES = True
```

И запускаешь. Это уже быстро — час-полтора обычно.

Если добавил новые бренды

Полный прогон, как при развёртывании с нуля. Старые данные при этом не побьются — все вставки в базу идут через `ON CONFLICT DO UPDATE`, так что если что-то уже было — оно просто обновится, а не задублируется.

Если хочешь запустить только один шаг

Любой скрипт можно дёрнуть напрямую без оркестратора:

```
python dimbrand.py
python Обновлпервич.py 2025
python енрич.py
python вольюмс.py 2025
```

Шаги 1 и 4 принимают год аргументом — без него возьмут значение по умолчанию из кода.

Параметры в `запук.py`

Помимо флагов запуска есть ещё пара настроек алгоритма. Менять их обычно не приходится, но если что:

PINGS_TOTAL — сколько проверок делает скрипт на одну модель в Шаге 1. Сейчас 90. Чем больше — тем точнее определяется производитель, но и медленнее.

SAMPLE_POSITIONS — на сколько точек разбивается выборка по году. Сейчас 6. Это нужно для равномерности — чтобы не брать все 90 проверок подряд из одного куска списка.

THRESHOLD — порог процента первичных проверок, при котором организация считается производителем. Сейчас 70%. Если кажется что в результаты попадают «не те» — попробуй сдвинуть в большую сторону.

Что в итоге окажется в базе

Четыре таблицы:

`dim_brands` — список брендов со страной и городом производителя. Маленькая таблица, по одной строке на бренд.

`manufacturers_verifiers` — кто чей производитель в каком году. Может быть несколько записей на одну модель, если её делают разные заводы.

`dim_reference` — характеристики моделей. Тут одна модель может занимать несколько строк — по одной на каждую категорию, если у неё бывают и бытовые диаметры и домовые например.

`production_volumes_exact` — главная таблица. Объёмы проверок по месяцам, разбитые по категориям. Структура: одна строка на каждую комбинацию (рег.номер + поверитель + год + месяц + категория). Есть колонка `точность` — она показывает, насколько надёжно посчитано это число. Если стоит `точно` — значит API отдало все данные и каждая проверка классифицирована индивидуально. Если что-то типа `пропорция_из_выборки` или `из_выборки` — значит API где-то падало и данные восстанавливались через страховочные механизмы.

Подробнее про каждую таблицу — в `README.md`, там расписана вся схема и связи между таблицами.

Если что-то идёт не так

Не подключается к базе. Проверь что PostgreSQL запущен и что настройки подключения в скриптах правильные. Самая частая причина — забыл поправить `PG = dict(...)` под свою базу.

Ругается на отсутствие какой-то библиотеки. Запусти `pip install -r requirements.txt` ещё раз. Бывает что не все зависимости встают с первого раза.

API возвращает 500. Это нормальная ситуация — реестр периодически отваливается, особенно на запросах с фильтром по дате. В коде уже есть страховка: данные за упавшие месяцы восстанавливаются через альтернативную выборку и помечаются в колонке `точность`. Если падает слишком часто и долго — отложи на пару часов и запусти снова.

Прогон идёт уже сутки и не заканчивается. Если это полный прогон — может быть

нормально, особенно если у тебя много брендов с большими объёмами проверок. Если ежемесячное обновление — что-то явно зависло. Прерви (`Ctrl+C`), посмотри в логах на каком производителе встал, проверь его руками в API. Перезапустить можно безопасно — благодаря идемпотентным апсертам ничего не задублируется.

Слишком много записей со значением `не_определено` в колонке точности. Значит у каких-то моделей не получилось автоматически определить диаметры — обычно потому что появился новый формат записи в API или модель совсем экзотическая. Найди их SQL-запросом, посмотри в `dim_reference`, заполни ручную колонку `ручное_исправление`. На следующем прогоне эти записи уже корректно распарсятся.

Когда что запускать

Удобный график работы:

- **Раз в месяц** (обычно первого числа) — ежемесячное обновление, только Шаг 4 за текущий год. Час-полтора.
 - **Раз в полгода** — полный прогон всех шагов. На ночь.
 - **Когда добавляешь нового конкурента** — полный прогон.
 - **Если что-то упало в середине** — повторить упавший шаг. Если это шаг 4 — можно даже сразу с того же года.
-

Если что-то осталось непонятным после этой инструкции — лезь в код, там везде есть комментарии. Ну или открой `README.md`, там подробнее про архитектуру и логику алгоритмов.

Удачи.